```scala
package com.citi.rockolympus.dataload

import com.citi.olympus.stepfrmwrk.common.OlympusBatchApplicationStep
import com.citi.olympus.stepfrmwrk.connector.common._
import com.citi.olympus.stepfrmwrk.util._
import org.apache.spark.sql._
import org.apache.spark.sql.expressions.Window
import org.apache.spark.sql.functions._
import org.slf4j.LoggerFactory
import org.apache.spark.sql.types._
import org.apache.spark.sql.Row
import org.apache.spark.storage.StorageLevel

import java.time.{Instant, LocalDate, LocalDateTime}
import java.time.format.DateTimeFormatter
import scala.collection.mutable.HashMap
import scala.util.Random
import java.sql.Timestamp
import java.text.SimpleDateFormat
import java.sql.{Connection, DriverManager, Timestamp}


class MMManualNetDelDataLoad extends  OlympusBatchApplicationStep {

  private val logger = LoggerFactory.getLogger(classOf[MMManualNetDelDataLoad])

  val name = "com.citi.rockolympus.dataload.MMManualNetDelDataLoad"
  val requiredCachedTables: List[String] = List.empty
  var moduleName: String = "MMManualNetDelDataLoad"
  val appId = SparkUtil.getSparkApplicationId()
  val requiredAdditionalContextVariables: Array[String] = Array.empty
  val description: String = "Data load for Manual Net delete table"
  val isIdempotent: Boolean = false
  val random = new Random()

  var busDate: String = _
  var spark: SparkSession = SparkUtil.spark
```

```
val processName:String = "MM_NetDelt"
val rckRawDb = "gfolyrck_raw"
val rckMngDb = "gfolyrck_managed"
var ManualNetDelInstance: String = "om_rck_mm_burndown_manualnetdel"
val processDetailTable = "om_rck_audit_prcs_dtl"
var srcDwhBusinessDate: String = ""
//var srcDwhUpdatedTime: String = ""
var srcDwhUpdatedTime: String = "NULL"
// Fixed millisecond-precision formatter for src_dwh_updated_time (matches parse pattern at write time)
private val srcTsFormatter = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss.SSS")
val MM_AUDIT_LOG_TABLE = "EFMUDMCC.OM_RCK_AUDIT_LOG_DTL"
val MM_AUDIT_LOG_TABLE_STG = "EFMUDMCC.OM_RCK_AUDIT_LOG_DTL_STG"
val MM_PROCESS_DETAIL_TABLE = "EFMUDMCC.OM_RCK_PRCS_DTL"
val MM_PROCESS_DETAIL_TABLE_STG = "EFMUDMCC.OM_RCK_PRCS_DTL_STG"
val THEME        = "AM BAU"
var maxStartTime: java.sql.Timestamp = _
var maxValues: Array[Row] = _
var maxBusinessDate: DataFrame = _
var manualNetDelItem: DataFrame = _
val hiveConnector = Connector.getConnector(ApplicationContext.hive_loader)


var loadStartTime : String = _
var loadEndTime : String = _
var lastLoadDate: String = ""
var preValidationCount:Long = 0
var postValidationCount:Long = 0
val author:String = "gfolyrck"
//val auditLogTable="rock_audit_log"
val randNum = random.nextInt(200).toString
val runID =  DateTimeFormatter.ofPattern("yyyyMMddHHmmSSS").format(LocalDateTime.now).concat(randNum).toLong
val dwh_business_date = DateTimeFormatter.ofPattern("yyyyMMdd").format(LocalDateTime.now).toInt
var activeJobs : Long =_

// Oracle connection details
var oracleUrl: String = _
var oracleUsername: String =
```

```
class MMManualNetDelDataLoad extends  OlympusBatchApplicationStep {
  // Oracle connection details
  var oracleUrl: String = _
  var oracleUsername: String = _
  var oraclePassword: String = _
  var oraclejdbcDriver: String = _
  var cobDate         : java.sql.Date = _
  var runTime         : Timestamp     = _
  var runDate         : String        = _
  var environmentName : String        = _

  private def requireProperty(key: String): String = {
    val value = ApplicationContext.getProperty(key)
    require(value != null && value.nonEmpty, s"Missing required config property: '$key'")
    value
  }

  private def initOracleConfig(): Unit = {
    oracleUrl = requireProperty("olycom_rck_oracleConf_jdbc_url")
    oracleUsername = requireProperty("olycom_rck_oracleConf_jdbc_user")
    oraclePassword = requireProperty("olycom_rck_oracleConf_jdbc_enc_password")
    oraclejdbcDriver = requireProperty("jdbc_driverClass")
    environmentName = ApplicationContext.getProperty("environmentProperty").trim

  }

  private def executeJdbcTransaction(insertSQL: String, label: String): Unit = {
    var connection : Connection         = null
    var stmt       : java.sql.Statement = null
    try {
      Class.forName(oraclejdbcDriver)
      connection = DriverManager.getConnection(oracleUrl, oracleUsername, oraclePassword)
      connection.setAutoCommit(false)
      stmt = connection.createStatement()
      logger.info(s"[$THEME Recon] Executing INSERT for $label | batch_run_id=$runID")
      stmt.execute(insertSQL)
      logger.info(s"[$THEME Recon] INSERT complete for $label.")
      connection.commit()
      logger.info(s"[$THEME Recon] Transaction committed for $label.")
```

lastLoadDate Aa ab .* 10 of 11

```
    } catch {
      case ex: Exception =>
        logger.error(s"[$THEME Recon] Transaction failed for $label: ${ex.getMessage}", ex)
        if (connection != null) {
          try { connection.rollback() } catch { case _: Exception => () }
          logger.info(s"[$THEME Recon] Transaction rolled back for $label.")
        }
        throw ex
    } finally {
      if (stmt       != null) try { stmt.close()      } catch { case _: Exception => () }
      if (connection != null) try { connection.close() } catch { case _: Exception => () }
    }
  }

  private def cleanStgForBatch(tableName: String,runID: Long): Unit = {
    var connection : Connection         = null
    var stmt       : java.sql.Statement = null
    try {
      Class.forName(oraclejdbcDriver)
      connection = DriverManager.getConnection(oracleUrl, oracleUsername, oraclePassword)
      connection.setAutoCommit(false)
      stmt = connection.createStatement()
      val deleteSQL = s"DELETE FROM $tableName WHERE RUN_ID = $runID"
      logger.info(s"[$THEME Recon] Truncating STG table: $tableName")
      stmt.execute(deleteSQL)
      connection.commit()
      logger.info(s"[$THEME Recon] STG truncate complete: $tableName")
    } catch {
      case ex: Exception =>
        logger.error(s"[$THEME Recon] STG truncate failed for $tableName: ${ex.getMessage}", ex)
        if (connection != null) try { connection.rollback() } catch { case _: Exception => () }
        throw ex
    } finally {
      if (stmt       != null) try { stmt.close()      } catch { case _: Exception => () }
      if (connection != null) try { connection.close() } catch { case _: Exception => () }
```

```
146     }
147   }
148
149   private def executeReconMergeAndInsert(runID: Long): Unit = {
150
151     val reconInsertSQL =
152       s"""
153          |INSERT INTO $MM_AUDIT_LOG_TABLE (
154          |    JOB_NAME,TABLE_NAME,LOAD_START_TIME,LOAD_END_TIME,
155          |    PRE_VALIDATION_COUNT,POST_VALIDATION_COUNT,STATUS,
156          |    AUTHOR,RUN_ID,DWH_BUSINESS_DATE,ENVIRONMENT
157          |)
158          |SELECT *
159          |FROM (
160          |    SELECT JOB_NAME,TABLE_NAME,LOAD_START_TIME,LOAD_END_TIME,
161          |    PRE_VALIDATION_COUNT,POST_VALIDATION_COUNT,STATUS,
162          |    AUTHOR,RUN_ID,DWH_BUSINESS_DATE,ENVIRONMENT
163          |    FROM $MM_AUDIT_LOG_TABLE_STG WHERE RUN_ID = $runID AND LOAD_END_TIME IS NOT NULL
164          |    ORDER BY LOAD_END_TIME DESC
165          |)
166          |WHERE ROWNUM = 1
167          |""".stripMargin
168
169     executeJdbcTransaction(reconInsertSQL, "AM AUDIT TABLE CHK")
170   }
171
172   private def executeReconMergeAndInsertProcess(runID: Long): Unit = {
173
174     val ProcessInsertSQL =
175       s"""
176          |INSERT INTO $MM_PROCESS_DETAIL_TABLE (
177          |    PROCESS_NAME,RUN_START_DATE,RUN_END_DATE,RUN_STATUS,SRC_DWH_BUSINESS_DATE,SRC_DWH_UPDATED_TIME,RUN_ID,DWH_BUSINESS_DATE,ENVIRONMENT
178          |)
179          |SELECT *
180          |FROM (
181          |    SELECT PROCESS_NAME,RUN_START_DATE,RUN_END_DATE,RUN_STATUS,SRC_DWH_BUSINESS_DATE,SRC_DWH_UPDATED_TIME,RUN_ID,DWH_BUSINESS_DATE,ENVIRONMENT
```

```
|FROM (
|    SELECT PROCESS_NAME,RUN_START_DATE,RUN_END_DATE,RUN_STATUS,SRC_DWH_BUSINESS_DATE,SRC_DWH_UPDATED_TIME,RUN_ID,DWH_BUSINESS_DATE,ENVIRONMENT
|    FROM $MM_PROCESS_DETAIL_TABLE_STG WHERE RUN_ID = $runID AND RUN_END_DATE IS NOT NULL
|    ORDER BY RUN_END_DATE DESC
|)
|WHERE ROWNUM = 1
|""".stripMargin

  executeJdbcTransaction(ProcessInsertSQL, "MM PROCESS TABLE CHK")
}

override def preProcess(context: ApplicationContext): Unit = {
  try{
    logger.info("Starting preprocessing...")

    spark.conf.set("spark.sql.caseSensitive", "false")
    spark.conf.set("spark.sql.files.ignoreCorruptFiles", "true")
    spark.conf.set("hive.exec.dynamic.partition", "true")
    spark.conf.set("hive.exec.dynamic.partition.mode", "nonstrict")
    spark.conf.set("hive.merge.mapfiles", "true")
    spark.conf.set("hive.merge.mapredfiles", "true")
    spark.conf.set("hive.merge.size.per.task", "256000000")
    spark.conf.set("hive.exec.compress.output", "true")
    spark.conf.set("parquet.block.size", "268435456")
    spark.conf.set("hive.exec.max.dynamic.partitions", 5000)
    spark.conf.set("hive.exec.max.dynamic.partitions.pernode", 5000)
    spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024)
    spark.conf.set("spark.sql.shuffle.partitions", 1000)
    spark.conf.set("spark.sql.adaptive.enabled", "true")
    spark.conf.set("spark.sql.adaptive.shuffle.targetPostShuffleInputSize", "32MB")
    spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
    spark.conf.set("spark.sql.broadcastTimeout", 600)
    spark.conf.set("spark.sql.parquet.filterPushdown", "true")
    spark.conf.set("spark.sql.orc.filterPushdown", "true")
    spark.conf.set("spark.sql.parquet.compression.codec", "snappy")
```

```
try{
    spark.conf.set("spark.sql.parquet.compression.codec", "snappy")
    spark.conf.set("spark.sql.inMemoryColumnarStorage.compressed", "true")
    spark.conf.set("spark.sql.inMemoryColumnarStorage.batchSize", "5000")
    spark.conf.set("spark.sql.ansi.enabled", false)
    spark.conf.set("spark.sql.join.preferSortMergeJoin", "true")
    spark.conf.set("spark.sql.legacy.timeParserPolicy", "LEGACY")

    // Oracle connection details
    initOracleConfig()

    val runDateTime = LocalDateTime.now()
    runTime         = Timestamp.valueOf(runDateTime)
    runDate         = runDateTime.format(DateTimeFormatter.ofPattern("yyyy-MM-dd"))
    cobDate         = java.sql.Date.valueOf(runDate)

    var retryCount = 0
    var maxRetires = 5

    val row = spark.read.format("jdbc")
      .option("url",oracleUrl)
      .option("dbtable",s"(SELECT COUNT(*) AS check_count FROM $MM_AUDIT_LOG_TABLE_STG WHERE status ='IN_PROGRESS' AND job_name = '$name' AND environment
      .option("user",oracleUsername)
      .option("password",oraclePassword)
      .option("driver",oraclejdbcDriver)
      .load()
      .collect()(0)

    activeJobs = row.getDecimal(0).longValue()

    while(activeJobs > 0 && retryCount < maxRetires)
    {
      logger.warn("previous batch is still running. retying in 1 minute...(Attempt:$retryCount)")
      Thread.sleep(600000)
      retryCount +=1

      val row = spark.read.format("jdbc")
        .option("url",oracleUrl)
```

```
    .option("user",oracleUsername)
    .option("password",oraclePassword)
    .option("driver",oraclejdbcDriver)
    .load()
    .collect()(0)

  activeJobs = row.getDecimal(0).longValue()

}

if(activeJobs > 0){
  logger.error("Previous Job did not finish after $maxRetires retries.Marking it as FAILED.")

  val sourceDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_AUDIT_LOG_TABLE_STG)
    .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()

  maxStartTime = sourceDf.filter(
      col("job_name") === lit(name) &&
        col("environment") === lit(environmentName)).agg(max(col("LOAD_START_TIME")).as("max_start_time"))
    .collect()(0).getTimestamp(0)

  val markFailedSQL = sourceDf.filter(col("LOAD_START_TIME") === lit(maxStartTime))
    .select(
      lit(name).as("job_name"),
      lit(ManualNetDelInstance).as("table_name"),
      col("load_start_time"),
      current_timestamp().as("load_end_time"),
      lit(preValidationCount).as("pre_validation_count"),
      lit(postValidationCount).as("post_validation_count"),
      lit("FAILED").as("status"),
      lit(author).as("author"),
      lit(runID).as("run_id"),
      lit(cobDate).cast(DateType).as("dwh_business_date"),
      lit(environmentName).as("environment")
```

```
287         )
288
289         markFailedSQL.write
290           .format("jdbc")
291           .option("url", oracleUrl)
292           .option("dbtable", MM_AUDIT_LOG_TABLE_STG)
293           .option("user", oracleUsername)
294           .option("password", oraclePassword)
295           .option("driver", oraclejdbcDriver)
296           .mode(SaveMode.Append)
297           .save()
298
299         executeReconMergeAndInsert(runID)
300
301         cleanStgForBatch(MM_AUDIT_LOG_TABLE_STG,runID)
302
303
304       }else {
305
306         cleanStgForBatch(MM_AUDIT_LOG_TABLE_STG, runID)
307
308
309
310         val insertAuditSQL = spark.range(1)
311           .select(
312             lit(name).as("job_name"),
313             lit(ManualNetDelInstance).as("table_name"),
314             current_timestamp().as("load_start_time"),
315             lit(null).cast(DataTypes.TimestampType).as("load_end_time"),
316             lit(null).cast("bigint").as("pre_validation_count"),
317             lit(null).cast("bigint").as("post_validation_count"),
318             lit("IN_PROGRESS").as("status"),
319             lit(author).as("author"),
320             lit(runID).as("run_id"),
```

```
320         lit(runID).as("run_id"),
321         lit(cobDate).cast(DateType).as("dwh_business_date"),
322         lit(environmentName).as("environment")
323
324       )
325
326     insertAuditSQL.write
327       .format("jdbc")
328       .option("url", oracleUrl)
329       .option("dbtable", MM_AUDIT_LOG_TABLE_STG)
330       .option("user", oracleUsername)
331       .option("password", oraclePassword)
332       .option("driver", oraclejdbcDriver)
333       .mode(SaveMode.Append)
334       .save()
335
336     logger.info("preprocessing completed successfully.")
337
338   }
339
340   cleanStgForBatch(MM_PROCESS_DETAIL_TABLE_STG, runID)
341
342   val processDetailsDF = spark.range(1)
343     .select(
344       lit(processName).as("process_name"),
345       current_timestamp().as("run_start_date"),
346       lit(null).cast(DataTypes.TimestampType).as("run_end_date"),
347       lit("IN_PROGRESS").as("run_status"),
348       (if (srcDwhBusinessDate == null || srcDwhBusinessDate == "NULL" || srcDwhBusinessDate.trim.isEmpty)
349         lit(null).cast(DateType)
350       else
351         to_date(lit(srcDwhBusinessDate), "yyyyMMdd")).as("src_dwh_business_date"),
352       (if (srcDwhUpdatedTime == null || srcDwhUpdatedTime == "NULL" || srcDwhUpdatedTime.trim.isEmpty)
353         lit(null).cast(DataTypes.TimestampType)
```

```
        lit(null).cast(DataTypes.TimestampType)
      else
        to_timestamp(lit(srcDwhUpdatedTime), "yyyy-MM-dd HH:mm:ss.SSS")).as("src_dwh_updated_time"),
      lit(runID).as("run_id"),
      lit(cobDate).cast(DateType).as("dwh_business_date"),
      lit(environmentName).as("environment")
    )
    .distinct()

  processDetailsDF.write
    .format("jdbc")
    .option("url", oracleUrl)
    .option("dbtable", MM_PROCESS_DETAIL_TABLE_STG)
    .option("user", oracleUsername)
    .option("password", oraclePassword)
    .option("driver", oraclejdbcDriver)
    .mode(SaveMode.Append)
    .save()

  logger.info("preprocessing completed successfully for process details table.")

  val processDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_PROCESS_DETAIL_TABLE)
    .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()


  val lastLoadedDateRow = processDf
    .filter(
      col("process_name") === lit(processName) &&
        col("run_status") === "SUCCESS" &&
        col("src_dwh_business_date").isNotNull
    )
    .orderBy(col("run_end_date").desc)
    .select(max("src_dwh_business_date").as("max_src_dwh_business_date"))
  //.first()

```

```
388         // Extract the lastLoadDate
389         // lastLoadDate = Option(lastLoadedDateRow).map(_.getAs[Int]("max_src_dwh_business_date").toString)
390         //  .getOrElse("NULL")
391
392         lastLoadDate = {
393           val rows = lastLoadedDateRow.collect()
394           if (rows.nonEmpty && !rows.head.isNullAt(rows.head.fieldIndex("max_src_dwh_business_date"))) {
395             rows.head.getAs[Int]("max_src_dwh_business_date").toString
396           } else {
397             "NULL"
398           }
399         }
400       logger.info("preprocessing completed successfully.")
401     }catch{
402       case e : Exception =>
403         logger.error("Error in Preprocessing:",e)
404         throw e
405     }
406   }
407
408
409
410   override def process(context: ApplicationContext): Unit = {
411
412
413     try {
414       logger.info("Starting Data Processing...")
415
416       // Oracle connection details
417       initOracleConfig()
418
419       val dateFormatter: DateTimeFormatter = DateTimeFormatter.ofPattern("yyyyMMdd")
420       val today = LocalDate.now()
421
422
```

```
422
423     val processMetadataDF = spark.table("gfolyrck_managed.mm_rock_strg_rpt_ctrl_rfrsh")
424       .filter(col("process_nm") === "NetDelete")
425       .select("start_dt", "end_dt", "active_flg")
426
427     /*val activeFlag = processMetadataDF.selectExpr("active_flg").collect()(0).getString(0)
428     val startDate: LocalDate = LocalDate.parse(processMetadataDF.selectExpr("start_dt").first().getInt(0).toString, dateFormatter)
429     val endDate: LocalDate = LocalDate.parse(processMetadataDF.selectExpr("end_dt").first().getInt(0).toString, dateFormatter)
430
431     val (currentStartDate, currentEndDate) = if (activeFlag.equalsIgnoreCase("y")) {
432       (startDate, endDate)
433     } else {
434       val formattedDate = LocalDate.now().format(dateFormatter)
435       (startDate, LocalDate.parse(formattedDate, dateFormatter))
436     }*/
437
438     if (processMetadataDF.isEmpty) {
439       throw new RuntimeException("Control data not found for process 'NetDelete'.")
440     }
441
442     val metadataRow = processMetadataDF.first()
443     val activeFlag = metadataRow.getAs[String]("active_flg")
444     val startDateStr = metadataRow.getAs[Int]("start_dt").toString
445     val endDateStr = metadataRow.getAs[Int]("end_dt").toString
446
447     if (startDateStr == "NULL" || endDateStr == "NULL") {
448       throw new IllegalArgumentException(s"Start date or end date is NULL in the control table. Start: '$startDateStr', End: '$endDateStr'")
449     }
450
451     val startDate: LocalDate = LocalDate.parse(startDateStr, dateFormatter)
452     val endDate: LocalDate = LocalDate.parse(endDateStr, dateFormatter)
453
454
455     val (currentStartDate, currentEndDate) = if (activeFlag.equalsIgnoreCase("y")) {
456       (startDate, endDate)
457     } else {
```

```
    (startDate, endDate)
  } else {
    val formattedDate = LocalDate.now().format(dateFormatter)
    (startDate, LocalDate.parse(formattedDate, dateFormatter))
  }


  val startStr: String = currentStartDate.format(dateFormatter)
  val endStr: String = currentEndDate.format(dateFormatter)


  val mainTable = spark.table("gfolygru_managed.om_ledger_item_audit_fact").alias("main")

  val requiredColumns = Seq("item_audit_id",   "instance_name",   "bank_category_code",   "bank_local_acc_no",   "item_bran_code",   "item_stmt_date",
    "item_item_id",   "item_ls_type",   "item_currency",   "bus_audit_action_timestamp",   "item_sys_entry_date",   "bus_audit_user_token",   "bus_audit
    "row_type",   "item_amount", "item_amount_6",   "item_flag_2",   "item_corr_acc_no",   "mf_sub_acc_no",   "bus_audit_action_date",   "bus_audit_orig_
    "item_sfield_7",   "item_sfield_8",   "item_sfield_9",   "item_string_1",   "item_string_10",   "item_string_11",   "item_string_12",   "item_string
    "item_string_17",   "item_string_18",   "item_string_19",   "item_string_20",   "item_string_21",   "item_string_22",   "item_string_23",   "item_st
    "item_string_31",   "item_string_32",   "item_string_33",   "item_string_34",   "item_string_35",   "item_string_36",   "item_string_37",   "item_st
    "item_string_44",   "item_string_45",   "item_string_46",   "item_string_47",   "item_string_48",   "item_string_49",   "item_string_5",   "item_str
    "item_string_57",   "item_string_58",   "item_string_59",   "item_string_6",   "item_string_60",   "item_string_61",   "item_string_62",   "item_str
    "item_string_7",   "item_string_8",   "item_string_9",   "item_stock_id",   "note",   "item_stock_type",   "item_flag_55",   "item_reference",   "dwh_
    // --- 87 NEW COLUMNS ADDED ---
    "TRANSACTION_QUANTITY", "TRANSACTION_DIRECTION", "ADDITIONAL_POSTING_INFORMATION_2", "USER_TRANSACTION_REFERENCE", "ENTRY_REFERENCE", "OUR_TRANSACTI
    "ADDITIONAL_POSTING_INFORMATION", "ISIN", "CUSTODIAN_BIC", "DEPOT_CODE", "BROKER_DEALER_NAME", "SUB_CUSTODIAN", "PRODUCT_PROCESSOR_CODE",
    "COUNTER_PARTY_ID", "MESSAGE_NUMBER", "SELL_AMOUNT", "EXCHANGE_RATE", "MIGRATION_LEGACY_MATCH_ID", "MIGRATION_LEGACY_ITEM_ID",
    "WRITE_OFF_ORIGINAL_ITEM_ID", "BUY_AMOUNT", "BUSINESS_ENTITY_CODE", "BUY_CURRENCY", "SELL_CURRENCY", "WRITE_OFF_DATE", "RECORD_DATE",
    "settlement_date", "AUTH_REVIEW_DEADLINE", "ALLOCATION_DATE", "PREVIOUS_ALLOCATION_DATE", "SWIFT_MESSAGE_TYPE", "SENDER_ACCOUNT_NUMBER",
    "PP_BRANCH_CODE", "NOMINEE_ACCOUNT", "TRANSACTION_CODE", "VALUE_DATE_MMYY", "STATEMENT_DATE_MMYY", "AI_MATCH_NUMBER", "AI_MATCH_TYPE",
    "AI_CONFIDENCE_LEVEL", "MIGRATION_MATCH_TYPE_FLAG", "WRITE_OFF_FLAG", "ALLOCATION_COUNT", "FREE_RETURN_COUNT", "ALLOCATION_OVERDUE_COUNT",
    "MATCH_REASON_CODE", "NON_ZERO_VALUES_IN_AMOUNT", "GL_ACCOUNT_NUMBER", "PAYMENT_REFERENCE_ID", "PAYMENT_TYPE", "SECURITY_NAME",
    "COUNTRY_CODE_ISO3", "EPT_PRODUCT_DESCRIPTION", "BENEFICIARY_ACCOUNT_NUMBER", "PREVIOUS_DEPARTMENT", "ALLOCATION_CODE", "CUSTOMER_ACCOUNT_MNEMONIC",
    "SECCORE_REFERENCE", "ORIGINATOR_BIC", "BROKER_DEALER_ID", "MARKET_MATCHING_REFERENCE", "MITI_REFERENCE", "SUPPLEMENTARY_REFERENCE", "branch_code",
    "TRADE_TYPE", "DATA_MESSAGE_DESTINATION", "DATA_MESSAGE_ORIGINATION", "MESSAGE_TYPE", "OUR_REFERENCE", "THEIR_REFERENCE", "item_entry_date", "item_i
    "item_sec_match_no", "item_stmt_item_ref", "item_stmt_no", "item_stmt_pg", "item_original_item_id", "item_owner", "item_last_action_owner", "item_la
    "item_exception_id", "item_short_no", "item_relationship_id", "item_fund_code", "dwh_business_date").map(c => s"main.$c")
```

```
"item_sec_match_no", "item_stmt_item_ref", "item_stmt_no", "item_stmt_pg", "item_original_item_id", "item_owner", "item_last_action_owner", "item_la
"item_exception_id", "item_short_no", "item_relationship_id", "item_fund_code", "dwh_business_date").map(c => s"main.$c")

manualNetDelItem =  if (startDate != null && endDate != null && activeFlag.equalsIgnoreCase("y"))
{

  mainTable.filter(
      col("dwh_business_date").between(lit(startStr).cast("int"), lit(endStr).cast("int")) &&
        col("bus_audit_operation_display_name") === "Net-Deleted" &&
        col("bus_audit_process_name") === "REC WC User Match"
    )
    .select(requiredColumns.map(col): _*)

} else {
  val tsInputFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss[.SSSSS][.SSS][.S]")

  // Guard against missing/invalid lastLoadDate (e.g. first run or no prior SUCCESS -> "NULL").
  // Fall back to the control-table start date so LocalDate.parse never receives "NULL".
  val prcsBusdate: LocalDate =
    if (lastLoadDate == null || lastLoadDate.trim.isEmpty || lastLoadDate.equalsIgnoreCase("NULL")) {
      logger.warn(s"lastLoadDate is '$lastLoadDate'; falling back to control-table start date $startStr for incremental window.")
      startDate
    } else {
      LocalDateTime.parse(lastLoadDate, tsInputFormatter).toLocalDate
    }

  val incStartStr = prcsBusdate.format(dateFormatter)
  val incEndStr = today.format(dateFormatter)

  mainTable.filter(
    col("dwh_business_date").between(lit(incStartStr).cast("int"), lit(incEndStr).cast("int")) &&
      col("bus_audit_operation_display_name") === "Net-Deleted" &&
      col("bus_audit_process_name") === "REC WC User Match"
  ).select(requiredColumns.map(col): _*)
```

```
524
525       }
526
527       maxBusinessDate = manualNetDelItem
528         .agg(max("dwh_business_date").as("max_dwh_business_date"))
529
530       // Preserve the real max business date; use "NULL" sentinel (NOT "0") when absent,
531       // because to_date("0", "yyyyMMdd") silently resolves to null.
532       srcDwhBusinessDate = {
533         val row = maxBusinessDate.first()
534         if (row != null && !row.isNullAt(row.fieldIndex("max_dwh_business_date"))) {
535           row.getAs[Int]("max_dwh_business_date").toString
536         } else {
537           "NULL"
538         }
539       }
540
541       maxValues = manualNetDelItem
542         .agg(
543           max("dwh_updated_time").as("max_dwh_updated_time")
544         )
545         .collect()
546
547       // Format explicitly to fixed millisecond precision so it matches the
548       // "yyyy-MM-dd HH:mm:ss.SSS" parse pattern used at write time. Timestamp.toString
549       // emits a variable-length fractional part which can fail to parse (-> null).
550       srcDwhUpdatedTime = if (maxValues.nonEmpty)
551         Option(maxValues(0).getAs[Timestamp]("max_dwh_updated_time"))
552           .map(ts => srcTsFormatter.format(ts))
553           .getOrElse("NULL")
554       else "NULL"
555
556       logging.info("Executing Source SQL : \nmanualNetDelItem")
557
558       logger.info(s"Pre-validation count: $preValidationCount records.")
559
```

```
23  class MMManualNetDelDataLoad extends  OlympusBatchApplicationStep {
410   override def process(context: ApplicationContext): Unit = {
413     try {
559
560       manualNetDelItem.createOrReplaceTempView("manualnetdel_vw")
561
562       val writeManualNetDelete = new HiveWriteProperties
563       writeManualNetDelete.sql = "select * from manualnetdel_vw"
564       writeManualNetDelete.databaseNm = "gfolyrck_managed"
565       writeManualNetDelete.tableNm = "om_rck_mm_burndown_manualnetdel"
566       writeManualNetDelete.partitionFlag = "Y"
567       writeManualNetDelete.partitionColumns = "dwh_business_date"
568       writeManualNetDelete.compactionFlag = "Y"
569       writeManualNetDelete.repartitionColumn="item_item_id"
570       //writeManualNetDelete.repartitionValue=2
571       writeManualNetDelete.overwriteFlag = "Y"
572
573       Connector.getConnector(context.hive_loader).write(writeManualNetDelete, scala.collection.mutable.HashMap.empty[String, String])
574
575       //val postValidationCount: Long = spark.sql(s"SELECT COUNT(*) FROM $rckMngDb.$ManualNetDelInstance").collect()(0)(0).toString.toLong
576
577       logger.info(s"Post-Validation count: $postValidationCount records.")
578
579       if (preValidationCount == postValidationCount) {
580         logger.info("Validation Successful : Record counts match.")
581       } else {
582         logger.warn(s"Validation Mismatch : pre-count:$preValidationCount ,post-count:$postValidationCount")
583       }
584
585
586       val sourceDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_AUDIT_LOG_TABLE_STG)
587         .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()
588
589       maxStartTime = sourceDf.filter(
590           col("job_name") === lit(name) &&
591             col("environment") === lit(environmentName)).agg(max(col("LOAD_START_TIME")).as("max_start_time"))
592         .collect()(0).getTimestamp(0)
593
594       val updateAuditSQL = sourceDf.filter(col("LOAD_START_TIME") === lit(maxStartTime))
```

lastLoadDate Aa ab .* 10 of 11

```
    val updateAuditSQL = sourceDf.filter(col("LOAD_START_TIME") === lit(maxStartTime))
      .select(
        lit(name).as("job_name"),
        lit(ManualNetDelInstance).as("table_name"),
        col("load_start_time"),
        current_timestamp().as("load_end_time"),
        lit(preValidationCount).as("pre_validation_count"),
        lit(postValidationCount).as("post_validation_count"),
        lit("SUCCESS").as("status"),
        lit(author).as("author"),
        lit(runID).as("run_id"),
        lit(cobDate).cast(DateType).as("dwh_business_date"),
        lit(environmentName).as("environment")
      )

    updateAuditSQL.write
      .format("jdbc")
      .option("url", oracleUrl)
      .option("dbtable", MM_AUDIT_LOG_TABLE_STG)
      .option("user", oracleUsername)
      .option("password", oraclePassword)
      .option("driver", oraclejdbcDriver)
      .mode(SaveMode.Append)
      .save()

    logger.info("Data Load Completed Successfully.")
  }catch{
    case e : Exception =>
      logger.error("Error during Data processing",e)

      // Oracle connection details
      initOracleConfig()

      val sourceDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_AUDIT_LOG_TABLE_STG)
```

```
627     val sourceDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_AUDIT_LOG_TABLE_STG)
628       .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()
629
630     maxStartTime = sourceDf.filter(
631         col("job_name") === lit(name) &&
632           col("environment") === lit(environmentName)).agg(max(col("LOAD_START_TIME")).as("max_start_time"))
633       .collect()(0).getTimestamp(0)
634
635     val failureSQL = sourceDf.filter(col("LOAD_START_TIME") === lit(maxStartTime))
636       .select(
637         lit(name).as("job_name"),
638         lit(ManualNetDelInstance).as("table_name"),
639         col("load_start_time"),
640         current_timestamp().as("load_end_time"),
641         lit(preValidationCount).as("pre_validation_count"),
642         lit(postValidationCount).as("post_validation_count"),
643         lit("FAILED").as("status"),
644         lit(author).as("author"),
645         lit(runID).as("run_id"),
646         lit(cobDate).cast(DateType).as("dwh_business_date"),
647         lit(environmentName).as("environment")
648       )
649
650     failureSQL.write
651       .format("jdbc")
652       .option("url", oracleUrl)
653       .option("dbtable", MM_AUDIT_LOG_TABLE_STG)
654       .option("user", oracleUsername)
655       .option("password", oraclePassword)
656       .option("driver", oraclejdbcDriver)
657       .mode(SaveMode.Append)
658       .save()
659
660     executeReconMergeAndInsert(runID)
```

```
662     cleanStgForBatch(MM_AUDIT_LOG_TABLE_STG,runID)
663
664     //Failed entry for process details table
665
666     val processDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_PROCESS_DETAIL_TABLE_STG)
667       .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()
668
669     val runStartDate = processDf.filter(
670       col("process_name") === lit(processName) &&
671         col("environment") === lit(environmentName)).agg(max("RUN_START_DATE").as("max_run_start_date"))
672       .collect()(0).getTimestamp(0)
673
674
675     val processDetailsDF = processDf.filter(col("RUN_START_DATE") === lit(runStartDate))
676       .orderBy(col("run_start_date").desc)
677       .limit(1)
678       .select(
679         col("process_name"),
680         col("run_start_date"),
681         current_timestamp().as("run_end_date"),
682         lit("FAILED").as("run_status"),
683         (if (srcDwhBusinessDate == null || srcDwhBusinessDate == "NULL" || srcDwhBusinessDate.trim.isEmpty)
684           lit(null).cast(DateType)
685         else
686         to_date(lit(srcDwhBusinessDate), "yyyyMMdd")).as("src_dwh_business_date"),
687         (if (srcDwhUpdatedTime == null || srcDwhUpdatedTime == "NULL" || srcDwhUpdatedTime.trim.isEmpty)
688           lit(null).cast(DataTypes.TimestampType)
689         else
690           to_timestamp(lit(srcDwhUpdatedTime), "yyyy-MM-dd HH:mm:ss.SSS")).as("src_dwh_updated_time"),
691         lit(runID).as("run_id"),
692         lit(cobDate).cast(DateType).as("dwh_business_date"),
693         lit(environmentName).as("environment")
694       )
695
696     processDetailsDF.write
```

```
696         processDetailsDF.write
697           .format("jdbc")
698           .option("url", oracleUrl)
699           .option("dbtable", MM_PROCESS_DETAIL_TABLE_STG)
700           .option("user", oracleUsername)
701           .option("password", oraclePassword)
702           .option("driver", oraclejdbcDriver)
703           .mode(SaveMode.Append)
704           .save()
705
706         // need to call execute and clean to stag
707         executeReconMergeAndInsertProcess(runID)
708
709         cleanStgForBatch(MM_PROCESS_DETAIL_TABLE_STG,runID)
710
711         throw e
712       }
713     }
714     override def postProcess(context: ApplicationContext): Unit = {
715
716       try{
717         logger.info("Starting Post-Processing...")
718
719         loadEndTime = Instant.now().toString
720
721         // Oracle connection details
722         initOracleConfig()
723
724         val runDateTime = LocalDateTime.now()
725         runTime         = Timestamp.valueOf(runDateTime)
726         runDate         = runDateTime.format(DateTimeFormatter.ofPattern("yyyy-MM-dd"))
727         cobDate         = java.sql.Date.valueOf(runDate)
```

```
cobDate          = java.sql.Date.valueOf(runDate)

val sourceDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_AUDIT_LOG_TABLE_STG)
  .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()

maxStartTime = sourceDf.filter(
    col("job_name") === lit(name) &&
      col("environment") === lit(environmentName)).agg(max(col("LOAD_START_TIME")).as("max_start_time"))
  .collect()(0).getTimestamp(0)

maxValues = manualNetDelItem
  .agg(
     max("dwh_business_date").as("max_dwh_business_date"),
     max("dwh_updated_time").as("max_dwh_updated_time")
    )
    .collect()

maxBusinessDate = manualNetDelItem
    .agg(max("dwh_business_date").as("max_dwh_business_date"))

      srcDwhBusinessDate = {
    val row = maxBusinessDate.first()
    if (row != null && !row.isNullAt(row.fieldIndex("max_dwh_business_date"))) {
      row.getAs[Int]("max_dwh_business_date").toString
    } else {
      "NULL"
    }
  }

  srcDwhUpdatedTime =
    if (maxValues.nonEmpty)
      Option(maxValues(0).getAs[Timestamp]("max_dwh_updated_time"))
        .map(ts => srcTsFormatter.format(ts))
        .getOrElse("NULL")
    else "NULL"
```

```scala
762
763     val successSQL = sourceDf.filter(col("LOAD_START_TIME") === lit(maxStartTime))
764       .select(
765         lit(name).as("job_name"),
766         lit(ManualNetDelInstance).as("table_name"),
767         col("load_start_time"),
768         current_timestamp().as("load_end_time"),
769         lit(preValidationCount).as("pre_validation_count"),
770         lit(postValidationCount).as("post_validation_count"),
771         lit("SUCCESS").as("status"),
772         lit(author).as("author"),
773         lit(runID).as("run_id"),
774         lit(cobDate).cast(DateType).as("dwh_business_date"),
775         lit(environmentName).as("environment")
776       )
777
778     successSQL.write
779       .format("jdbc")
780       .option("url", oracleUrl)
781       .option("dbtable", MM_AUDIT_LOG_TABLE_STG)
782       .option("user", oracleUsername)
783       .option("password", oraclePassword)
784       .option("driver", oraclejdbcDriver)
785       .mode(SaveMode.Append)
786       .save()
787
788
789     executeReconMergeAndInsert(runID)
790
791     cleanStgForBatch(MM_AUDIT_LOG_TABLE_STG,runID)
792
793     logger.info("Processing completed audit final table successfully.")
794
795     val processDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_PROCESS_DETAIL_TABLE_STG)
```

```
val processDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_PROCESS_DETAIL_TABLE_STG)
  .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()

val runStartDate = processDf.filter(
    col("process_name") === lit(processName) &&
      col("environment") === lit(environmentName)).agg(max("RUN_START_DATE").as("max_run_start_date"))
  .collect()(0).getTimestamp(0)

val processDetailsDF = processDf.filter(col("RUN_START_DATE") === lit(runStartDate))
  .orderBy(col("run_start_date").desc)
  .limit(1)
  .select(
    col("process_name"),
    col("run_start_date"),
    current_timestamp().as("run_end_date"),
    lit("SUCCESS").as("run_status"),
    (if (srcDwhBusinessDate == null || srcDwhBusinessDate == "NULL" || srcDwhBusinessDate.trim.isEmpty)
      lit(null).cast(DateType)
    else
      to_date(lit(srcDwhBusinessDate), "yyyyMMdd")).as("src_dwh_business_date"),
    (if (srcDwhUpdatedTime == null || srcDwhUpdatedTime == "NULL" || srcDwhUpdatedTime.trim.isEmpty)
      lit(null).cast(DataTypes.TimestampType)
    else
      to_timestamp(lit(srcDwhUpdatedTime), "yyyy-MM-dd HH:mm:ss.SSS")).as("src_dwh_updated_time"),
    lit(runID).as("run_id"),
    lit(cobDate).cast(DateType).as("dwh_business_date"),
    lit(environmentName).as("environment")
  )

processDetailsDF.write
  .format("jdbc")
  .option("url", oracleUrl)
  .option("dbtable", MM_PROCESS_DETAIL_TABLE_STG)
  .option("user", oracleUsername)
```

```
828         .option("user", oracleUsername)
829         .option("password", oraclePassword)
830         .option("driver", oraclejdbcDriver)
831         .mode(SaveMode.Append)
832         .save()
833
834       executeReconMergeAndInsertProcess(runID)
835
836       cleanStgForBatch(MM_PROCESS_DETAIL_TABLE_STG,runID)
837
838       logger.info("Post-Processing completed successfully.")
839
840     }catch{
841       case e : Exception =>
842         logger.info("Error Post-Processing",e)
843
844         // Oracle connection details
845         initOracleConfig()
846
847         val sourceDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_AUDIT_LOG_TABLE_STG)
848           .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()
849
850         maxStartTime = sourceDf.filter(
851             col("job_name") === lit(name) &&
852               col("environment") === lit(environmentName)).agg(max(col("LOAD_START_TIME")).as("max_start_time"))
853           .collect()(0).getTimestamp(0)
854
855         val failureSQL =sourceDf.filter(col("LOAD_START_TIME") === lit(maxStartTime))
856           .select(
857             lit(name).as("job_name"),
858             lit(ManualNetDelInstance).as("table_name"),
859             col("load_start_time"),
860             current_timestamp().as("load_end_time"),
861             lit(preValidationCount).as("pre validation count"),
```

```
862             lit(postValidationCount).as("post_validation_count"),
863             lit("FAILED").as("status"),
864             lit(author).as("author"),
865             lit(runID).as("run_id"),
866             lit(cobDate).cast(DateType).as("dwh_business_date"),
867             lit(environmentName).as("environment")
868
869           )
870
871         failureSQL.write
872           .format("jdbc")
873           .option("url", oracleUrl)
874           .option("dbtable", MM_AUDIT_LOG_TABLE_STG)
875           .option("user", oracleUsername)
876           .option("password", oraclePassword)
877           .option("driver", oraclejdbcDriver)
878           .mode(SaveMode.Append)
879           .save()
880
881         executeReconMergeAndInsert(runID)
882
883         cleanStgForBatch(MM_AUDIT_LOG_TABLE_STG,runID)
884
885
886         throw e
887       }
888     }
889
890     override def cleanupBeforeRerun(context: ApplicationContext): Unit ={
891       try{
892         logging.info("Starting cleanup before rerun")
893
894         // Oracle connection details
895         initOracleConfig()
```

```
895     initOracleConfig()
896
897     val sourceDf=spark.read.format("jdbc").option("url",oracleUrl).option("dbtable",MM_AUDIT_LOG_TABLE_STG)
898       .option("user",oracleUsername).option("password",oraclePassword).option("driver",oraclejdbcDriver).load()
899
900     maxStartTime = sourceDf.filter(
901         col("job_name") === lit(name) &&
902           col("environment") === lit(environmentName)).agg(max(col("LOAD_START_TIME")).as("max_start_time"))
903       .collect()(0).getTimestamp(0)
904
905     val updateRetrySQL = sourceDf.filter(col("LOAD_START_TIME") === lit(maxStartTime)
906         && col("status") === "FAILED" && col("dwh_business_date") === lit(cobDate).cast(DateType)
907       )
908       .select(
909         lit(name).as("job_name"),
910         lit(ManualNetDelInstance).as("table_name"),
911         col("load_start_time"),
912         current_timestamp().as("load_end_time"),
913         lit(preValidationCount).as("pre_validation_count"),
914         lit(postValidationCount).as("post_validation_count"),
915         lit("RETRY").as("status"),
916         lit(author).as("author"),
917         lit(runID).as("run_id"),
918         lit(cobDate).cast(DateType).as("dwh_business_date"),
919         lit(environmentName).as("environment")
920       )
921
922     updateRetrySQL.write
923       .format("jdbc")
924       .option("url", oracleUrl)
925       .option("dbtable", MM_AUDIT_LOG_TABLE_STG)
926       .option("user", oracleUsername)
927       .option("password", oraclePassword)
928       .option("driver", oraclejdbcDriver)
929       .mode(SaveMode.Append)
930       .save()
```

```
930         .save()
931
932       executeReconMergeAndInsert(runID)
933
934       cleanStgForBatch(MM_AUDIT_LOG_TABLE_STG,runID)
935       // cleanup before rerun
936       println(s"cleanup part completed successfully for process details table $srcDwhUpdatedTime $srcDwhBusinessDate")
937       logging.info("cleanup before rerun successfully.")
938     }catch{
939       case e : Exception =>
940         logger.info("Error in rerun before cleanup",e)
941         throw e
942     }
943   }
944 }
945
946
```